

Kịch bản triển khai đo đạc

Kịch bản triển khai đo đạc — WP2 / WP4 / WP5 trên hai nền tảng

Tiền đề: môi trường WP1 đã dựng theo README (verify_env PASS trên máy đang đứng).
Nguyên tắc: số đo là tài sản theo máy; chạy ĐỦ kịch bản trên TỪNG máy; so chéo máy chỉ bằng tỉ lệ.

0. Ba file mới bổ sung (vị trí + vai trò + nguồn)

File	Vị trí	Vai trò (mục đề bài)	Nguồn căn cứ
gen_tls_certs.sh	scripts/	Sinh chứng thư server RSA-2048 / ECDSA P-256 / ML-DSA-65 cho WP4 (§7.4 macro, Deliverables “TLS integration scripts”)	Tên thuật toán theo docs OpenSSL 3.5: EVP_PKEY-ML-DSA (“ML-DSA-44/65/87... added in OpenSSL 3.5”); KEM không nằm trong cert (openssl-users, V. Dukhovni)
bench_tls.sh	scripts/	Đo handshake TLS 1.3: latency (median/mean/p95, §7.5) + throughput đồng thời (§7.4) trên ma trận cert × group; xuất data/tls_handshake_25519MLKEM768 (\$9 network overhead qua cột cert_bytes)	Tên group theo man SSL_CTX_set1_groups_list 3.5: X25519MLKEM768 (hybrid, RQ3), MLKEM768; “DEFAULT list selects X25519MLKEM768 as one of the predicted keyshares”; s_server/s_client manpages (docs.openssl.org/3.5/man1)
analyze.py	scripts/	Gom mọi CSV → analyze-sis_out/tables.md + biểu đồ PNG (§9 Presentation: “charts for latency vs parameter set, throughput, bytes vs algorithm”; Tuần 11)	pandas không bắt buộc — chỉ csv chuẩn + matplotlib (docs.python.org/csv, matplotlib.org); schema khớp đúng các CSV của run_micro/measure_memory/mea

Makefile thêm hai target thuận cộng: make tls, make analyze (đồng bộ kiểu make bench/memory/codesize).

Hai mục đề bài KHÔNG có script — khai báo phạm vi (ghi vào Limitations, §13): - *Energy/power* (§7.4, §9): để chỉ rõ cần phần cứng đo riêng (Monsoon, INA219/226) — không có thiết bị ⇒ ngoài phạm vi, nêu rõ trong báo cáo. - *pqm4/MCU* (§7.2): điều kiện “if testing MCU targets” không kích hoạt — phạm vi là server x86_64 + SBC Cortex-A72; pqm4 trích dẫn ở Literature như chuẩn đo lớp MCU.

Bảng quy mức an toàn (dùng khi đọc kết quả — §“map tới mức bảo mật tương đương”):

Mức (FIPS 203/204 category)	PQC	RSA tương đương	EC tương đương
1 (~AES-128)	ML-KEM-512, ML-DSA-44	RSA-3072	P-256
3 (~AES-192)	ML-KEM-768, ML-DSA-65	RSA-7680	P-384
5 (~AES-256)	ML-KEM-1024, ML-DSA-87	RSA-15360	P-521

RSA-2048 (~112-bit, theo NIST SP 800-57) vẫn đo vì là baseline thực dụng phổ biến — khi so kết quả phải chú thích nó dưới mức 1.

1. NỀN TẢNG 1 — PC x86_64

1.1 Bắt buộc, theo thứ tự

```
source scripts/setenv.sh                                # mỗi terminal
sudo cpupower frequency-set -g performance             # §7.4 repeatability
make                                                    # build/bench_evp

# WP2 — microbenchmark, K=5 batch (§7.5 median-of-medians):
for i in 1 2 3 4 5; do
    make bench
    cp data/summary_micro_x86_64.csv data/raw/x86_64/summary_batch$i.csv
done

# WP5 — bộ nhớ + kích thước:
make memory
make codesize
bash scripts/fetch_pqclean.sh clean                    # nếu chưa: .a per-algorithm
size ~/pqclean/src/PQClean/crypto_*/ml-*/clean/lib*.a | tee data/raw/x86_64/pqclean

# WP3 — liboqs ref/opt ra CSV (tự bỏ qua cây chưa build; trên x86 ref là đủ)
bash scripts/run_liboqs_speed.sh

# WP4 — TLS handshake (cert x group):
bash scripts/gen_tls_certs.sh
make tls                                                # ~5-10 phút với mặc định

# Phân tích:
make analyze                                            # analysis_out/tables.md + *.p

# Đẩy bằng chứng ra ngoài:
git add data docs analysis_out && git commit -m "x86_64 measurements" && git push
```

Ghi chú K-batch: analyze.py đọc bản summary mới nhất; 5 file batch trong data/raw/ là bằng chứng §7.5 và đầu vào cho phân tích sâu (bootstrap CI) ở tuần tổng hợp.

1.2 Tùy chọn (thêm chiều dữ liệu)

```

bash scripts/build_liboqs.sh opt # hiệu ứng AVX2 (đã có mẫu x2.
bash scripts/fetch_pqclean.sh avx2 # giá code-size của tối ưu
CERT_SET="mldsa44 mldsa87 rsa3072" bash scripts/gen_tls_certs.sh # mở rộng
TLS_CONC=8 TLS_DUR=15 make tls # throughput tải nặng hơn
~/pqc/openssl/bin/openssl speed rsa2048 ecdsap256 # đối chứng A1
# Bytes thật trên dây của 1 handshake ($9 network overhead) — chạy ở terminal
# trong lúc terminal 1 bắt đúng 1 handshake bằng s_client:
sudo tshark -i lo -f "tcp port 4433" -a duration:5 -q -z conv,tcp

```

2. NỀN TẢNG 2 — ARM Raspberry Pi 4 (Mythic Beasts)

2.1 Bắt buộc, theo thứ tự

```

source scripts/setenv.sh
sudo cpupower frequency-set -g performance # Pi mặc định ondemand
# (thiếu cpupower: echo performance | sudo tee /sys/devices/system/cpu/cpu*/
make

# WP2 — K=5 batch, CÓ NGHĨ NGUỘI giữa batch (Pi 4 throttle ~80°C):
for i in 1 2 3 4 5; do
    make bench
    cp data/summary_micro_aarch64.csv data/raw/aarch64/summary_batch$i.csv
    cat /sys/class/thermal/thermal_zone0/temp # <70000 mỗi chạy batch kế
    sleep 120
done

# WP5:
make memory
make codesize
bash scripts/fetch_pqclean.sh clean
bash scripts/fetch_pqclean.sh aarch64 # trực NEON thử hai
size ~/pqc/src/PQClean/crypto_kem/ml-kem-768/{clean,aarch64}/lib*.a | tee da

# WP3 — TRỌNG TÂM trên máy này: cần CẢ HAI cây ref+opt đã build:
bash scripts/run_liboqs_speed.sh # -> data/liboqs_speed_aarch64

# WP4:
bash scripts/gen_tls_certs.sh # cert sinh LẠI trên máy này
make tls

make analyze
git add data analysis_out && git commit -m "aarch64 measurements" && git pus

```

2.2 Tùy chọn

```

USE_ARM_PMU=1 FORCE=1 bash scripts/build_liboqs.sh opt # CHỈ sau khi nạp m
FORCE=1 bash scripts/build_openssl.sh # full test suite,

```

2.3 Đặc thù Pi 4 cần nhớ

Cortex-A72 không có ARM Crypto Extensions (cập ref/opt sạch — delta = NEON). Mạng IPv6-only: GitHub đi qua NAT64/DNS64 của Mythic Beasts. Mọi phiên đo kết thúc bằng push — máy host có thể thu hồi.

3. Nghiệm thu hai nền tảng (tick đủ mới sang bước viết)

x86_64: [] 5 batch summary trong data/raw/x86_64 · [] memory/codesize CSV · [] pqclean size .txt · [] liboqs_speed_x86_64.csv ($\geq$ cây ref) · [] tls_handshake_x86_64.csv đủ ≥ 9 hàng (3 cert $\times$ 3 group) · [] analysis_out có tables.md + PNG · [] đã push

aarch64: [] 5 batch (kèm log nhiệt) · [] memory/codesize CSV · [] pqclean clean+aarch64 size · [] tls_handshake_aarch64.csv · [] liboqs_speed_aarch64.csv đủ CẢ ref+opt (12 thuật toán $\times$ ops) · [] analysis_out cập nhật cả hai máy (bảng Cross-platform ratio xuất hiện) · [] đã push trước khi trả máy

4. Deliverable 2 & 5 — Báo cáo và Demo

Báo cáo (PDF): khung LaTeX ở docs/report/main.tex, biên dịch bằng XeLaTeX (tiếng Việt):

```
cd docs/report && xelatex main.tex && xelatex main.tex # 2 lần cho mục lục
```

Khung đã đặt sẵn chỗ chèn biểu đồ từ analysis_out/ (tự ẩn khi file chưa tồn tại — biên dịch được ngay cả trước khi đo xong). Điền kết quả tới đâu, biên dịch tới đó.

Demo (quay màn hình ~2 phút):

```
bash scripts/demo.sh
```

Trình tự demo khớp nguyên văn Deliverable 5 (“benchmark runs and TLS handshake comparisons”): chứng minh môi trường $\rightarrow$ một microbenchmark sống (ML-KEM-768 vs RSA-2048) $\rightarrow$ so handshake X25519 / hybrid / pure-PQC với chứng thư ML-DSA. Quay trên x86 là đủ; bản ARM quay thêm nếu còn giờ thuê.

§7.6 (integration, “small HTTPS server”): đã được phủ bởi chính bench_tls.sh — openssl s_server là server TLS tối giản, đo end-to-end trên loopback; muốn thêm chiều “HTTP thật” thì chạy server với -www (phục vụ một trang trạng thái) và lặp lại phép đo — ghi vào báo cáo như mở rộng tùy chọn.

5. WP4 phương án nginx — server thật (khuyến nghị chạy KÈM s_server)

Vì sao thêm nginx: (1) §7.6 đòi “small HTTPS server” end-to-end — nginx là server thật; (2) §7.4 đòi đo “single-threaded **and** multi-threaded servers” — chính là worker_processes 1 vs auto; (3) hai bộ số s_server/nginx đối chứng nhau làm chặt phương pháp. Cốt lõi kỹ thuật: **nginx lấy toàn bộ TLS từ OpenSSL nó link** — nginx của apt link OpenSSL 3.0 (không ML-KEM) nên phải build từ nguồn link với 3.6.2 của ta; group PQC đi qua directive ssl_ecdh_curve (đã kiểm sống: nhận X25519MLKEM768:MLKEM768:X25519, từ chối group giả).

```
# Cả hai nền tảng, sau build_openssl + gen_tls_certs:
bash scripts/build_nginx.sh # x86 vài phút; Pi ~5-10 phút (KHÔNG re
bash scripts/bench_tls_nginx.sh # -> data/tls_handshake_nginx-<arch>.cs
make analyze # bảng/biểu đồ tự thêm nhãn nginx-<arch>
```

Đối chiếu Apache (config TLS 1.3 bữa trước) $\leftrightarrow$ nginx:

Apache	nginx
SSLEngine on	listen 443 ssl;
SSLProtocol -all +TLSv1.3	ssl_protocols TLSv1.3;
SSLCertificateFile/KeyFile	ssl_certificate/ ssl_certificate_key
SSLOpenSSLConfCmd Groups ...	ssl_ecdh_curve ...
SSLSessionTickets off	ssl_session_tickets off; (+ ssl_session_cache off;)

Bây đã kiểm/tra nguồn: TẤT resumption (cache+tickets) để mọi kết nối là full handshake — không tắt là TLS 1.3 resume bằng PSK, bỏ qua truyền chứng thư, số đo vô nghĩa; `ssl_ecdh_curve` đặt ở server block **default** (ticket nginx #2542: block name-based khác bị bỏ qua lặng lẽ — người báo lỗi vấp đúng khi bật PQC); `return` cần module rewrite (đã tắt để khỏi kéo PCRE) → template trả file tĩnh; `wrk` chỉ dùng được cho hàng X25519 (nó link OpenSSL hệ thống) — dùng như cross-check công cụ trên hàng đó, không dùng cho PQC.

6. Server HTTPS tự triển khai — `src/tls_mini_server.c` (§7.6, tăng tích hợp thứ ba)

Tự sở hữu TCP + HTTP, TLS 1.3-only cấu hình qua libssl (KHÔNG tự viết lại giao thức TLS — định luật dùng-tự-chế-crypto); mẫu theo OpenSSL `demos/guide/tls-server-block.c`; đồng hồ đo `CLOCK_MONOTONIC_RAW` quanh `SSL_accept` đặt Ở PHÍA SERVER (tinh thần `s_timer.c` của Paquin-Stebila-Tamvada nhưng đảo phía) — mỗi kết nối in một dòng CSV `conn,tls,group,handshake_us`, trong đó cột `group` là **bằng chứng cứng** kết nối thật sự dùng MLKEM768/X25519MLKEM768.

```
make tlsmini                                # build (link OpenSSL của ta,
source scripts/setenv.sh
./build/tls_mini_server ~/pqc/tls/mldsa65.cert.pem ~/pqc/tls/mldsa65.key.pem
| tee data/raw/$(uname -m)/tlsmini_handshakes.csv &
curl -k https://127.0.0.1:9443/              # HTTPS end-to-end -> "ok"
TLS_GROUPS=MLKEM768 ./build/tls_mini_server ... # ép thuần PQC cho một ph
```

Đã kiểm sống (sandbox): 3 kết nối HTTPS liên tiếp OK; client ép TLS 1.2 bị từ chối (alert protocol version); `TLS_GROUPS` bịa → từ chối khởi động với lỗi OpenSSL gốc; warm-up lộ rõ trong số (kết nối #1 ~8.0ms, #2-3 ~3.8ms) — đúng lý do mọi phép đo có vòng warm-up. Vai trò: bậc giữa của thang tích hợp **thư viện (bench_evpc)** → **server tự viết (file này)** → **server sản xuất (nginx)**; KHÔNG thay thế số `nginx/s_server`, mà bổ sung góc nhìn server-side và bằng chứng `group`.